

EmbedSim

EMBEDDED CONTROL · OPEN SOURCE

Visual Block Diagram Designer

A Student Guide for UI / Workflow Research

Architecture · The >> Syntax · Model–View–Controller · Web Extension

Based on: `embedsim_gui.py` · EmbedSim open-source framework
github.com/vectorsim/embed_sim_project

Table of Contents

1	What the GUI Is and Why It Exists	3
2	The >> Syntax — EmbedSim's Wiring Operator	5
3	Drag-and-Drop Block Placement	7
4	Code Generation — From Canvas to Python File	9
5	Running the Simulation	12
6	Architecture — Model · View · Controller	14
7	The Block Registry — Adding New Blocks	17
8	The Web Version — Design and Feasibility	19
9	Thesis Improvements — What to Build and Why	22
10	Deliverables and Thesis Claims	25
11	Your First Week — Day by Day	27
R	Reference: Key Classes, Methods and Libraries	29

1 What the GUI Is and Why It Exists

1.1 The Problem It Solves

Writing EmbedSim simulation code by hand looks simple for a single sine wave through a gain block. It becomes a transcription marathon for a PMSM motor controller with twelve blocks, seventeen wiring connections, and five parameters each. A developer who mis-types one variable name in the wiring section gets a runtime exception, not a compile error.

The GUI eliminates this class of error entirely. You assemble the simulation visually — dragging blocks, connecting pins — and the tool generates the Python file for you. The generated file is syntactically correct by construction.

1.2 What the Current Prototype Does

The prototype (`embedsim_gui.py`) is a desktop application built on Dear PyGui. It implements:

- A block palette with all registered EmbedSim block types, organised by category.
- A node-editor canvas where blocks become draggable nodes with input/output pins.
- Pin-to-pin wiring by dragging from an output pin to an input pin.
- A parameter sidebar — click any node to edit its parameters inline.
- An algebraic loop validator that runs a DFS cycle-detection algorithm and flags loops not broken by a VectorDelay.
- A code generator that produces a complete, runnable EmbedSim .py file.
- A save button that writes the generated file to disk.

1.3 What It Does NOT Yet Do

Understanding the current gaps is the starting point for your thesis. The prototype does not:

- Run the simulation from within the GUI — you must open a terminal and run the generated file.
- Display plots inside the application.
- **BLOCK_REGISTRY** Auto-discover new blocks — adding a new EmbedSim block class requires a manual entry in
- Persist the canvas to a file (no save/load of the diagram itself, only export to .py).
- Run in a web browser — it requires a desktop Python installation with Dear PyGui.



Your Thesis Scope in One Sentence

Design, implement, and evaluate a user interface and workflow for EmbedSim that lowers the barrier for IoT/AI engineers to create, run, and iterate on block diagram simulations — on the desktop and optionally in a web browser.

2 The >> Syntax — EmbedSim's Wiring Operator

2.1 What >> Actually Does

The >> symbol looks like it might be a bit-shift operator borrowed from C. In EmbedSim it means something entirely different: "connect the output of the left block to an input of the right block."

When Python evaluates `sin_src >> gain`, it calls `sin_src.__rshift__(gain)`. EmbedSim's `VectorBlock` base class defines this method to register the connection in the simulation's adjacency graph. No actual data flows at this point — you are only declaring the topology.

```
# These two lines are exactly equivalent:
sin_src >> gain           # human-readable, operator syntax
sin_src.__rshift__(gain) # what Python actually calls

# Chaining works too:
sin_src >> summer >> gain >> sink
# equivalent to:
sin_src >> summer
summer >> gain
gain >> sink
```

2.2 Why This Design Was Chosen

MATLAB/Simulink uses a graphical wiring editor — there is no text representation of connections. Simulink's text-based variant (MATLAB code) uses `set_param()` function calls, which are verbose and hard to read.

The >> operator gives EmbedSim simulations the readability of a schematic diagram. A developer reading the code can trace the signal flow left-to-right, exactly as they would trace it on a block diagram. This is the key design decision that makes EmbedSim files both human-writable and machine-parseable — which is precisely what enables the GUI's code generator.

Metaphor: the >> operator as a pipe

In Unix shell: `cat file.txt | grep error | wc -l`

Data flows left to right through a chain of processors.

In EmbedSim: `sin_src >> gain >> sink`

Signals flow left to right through a chain of blocks.

Both express a directed graph as a readable linear sequence.

2.3 Multi-Input Blocks — VectorSum

Some blocks take more than one input. `VectorSum` with `n_inputs=2` expects two upstream connections. The order in which you write them matters — the first >> connection is input 0, the second is input 1.

```
summer = VectorSum('summer', signs=[1, -1], n_inputs=2)

ref_signal >> summer    # input 0 – added
feedback   >> summer    # input 1 – subtracted (sign = -1)

# If you wired them the other way around, the subtraction would be wrong.
```

```
# The GUI preserves connection order – important for correctness.
```

2.4 Algebraic Loops — Why They Are a Problem

An algebraic loop occurs when a signal feeds back into itself without any delay. Imagine a thermostat that sets the heater power based on the temperature it is currently changing — the equation to solve at every time step becomes circular, with no unique solution.

```
# ALGEBRAIC LOOP – will crash or give wrong results:
summer >> gain >> summer      # gain feeds back to summer immediately

# CORRECT – broken by a one-step delay:
summer >> gain >> sink
gain  >> delay
delay >> summer                # delay outputs the PREVIOUS step's value
```

The GUI's Validate button runs a depth-first search on the connection graph. Any cycle that does not pass through a `vectorDelay` (a loop-breaker) is flagged as an algebraic loop with the names of the involved blocks.

3 Drag-and-Drop Block Placement

3.1 The Node Editor Model

The GUI uses Dear PyGui's node editor widget, which provides the block-and-wire canvas. Each EmbedSim block becomes a "node" in Dear PyGui's terminology. Each node has attributes — Dear PyGui's term for the input and output pins.

Think of it like a physical patch bay in a recording studio. Each device (node) has input jacks (mvNode_Attr_Input) and output jacks (mvNode_Attr_Output). You connect them by plugging a cable from an output jack to an input jack. The GUI's canvas is that patch bay.

3.2 What Happens When You Double-Click the Palette

The palette is a column of `dpg.add_selectable()` items. Each selectable's callback fires on double-click, passing the class name as its label. The callback chain is:

```
cb_palette_double_click(sender, app_data)
└─ reads class name from dpg.get_item_label(sender)
    └─ add_node(class_name, pos=(x, y))
        └─ creates NodeInst with uid, var_name, params
            └─ calls dpg.node(label=..., parent='node_editor', pos=pos)
                └─ adds input attribute (if not a source block)
                    └─ adds static attribute (parameter display text)
                        └─ adds output attribute (if not a sink block)
                            └─ registers attr_tag → uid in attr_to_node dict
```

The position `pos` is computed from the current count of nodes, arranged in a 4-column grid so blocks do not pile up on top of each other.

3.3 What Happens When You Draw a Wire

Dear PyGui handles the drag visually — you see a Bezier curve following the mouse. When you release on a valid input pin, Dear PyGui fires the `cb_link_created` callback with a tuple (`attr_out_tag`, `attr_in_tag`).

```
def cb_link_created(sender, app_data):
    a_out, a_in = app_data          # pin tags from Dear PyGui
    luid = _next_link()             # generate unique link ID
    links[luid] = (a_out, a_in)    # store in links dict
    dpg.add_node_link(a_out, a_in, parent='node_editor', tag=luid)
    _refresh_status()
```

The `links` dictionary maps link IDs to pin-tag pairs. The `attr_to_node` dictionary maps each pin tag back to the block's node ID. Together these two dictionaries give you the complete signal flow graph, which is used both by the code generator and the loop validator.

3.4 Editing Parameters in the Sidebar

Clicking a node fires the global mouse handler `_on_viewport_click`, which polls `dpg.get_selected_nodes()`. When exactly one node is selected, `show_sidebar(uid)` is called.

The sidebar clears any previous widgets, then builds a fresh set of input widgets for each parameter of the selected block. The widget type adapts to the parameter's Python type: `add_input_float` for

floats, `add_input_int` for integers, `add_checkbox` for booleans, and `add_input_text` for strings and lists.

Each widget's callback calls `_update_param(uid, pname, value)` which updates the `NodeInst.params` dictionary and refreshes the inline label on the canvas node. The canvas and sidebar always show the same values.

The Three Data Structures That Drive Everything

nodes: `uid` → `NodeInst` (class name, var name, params, pin tags)

links: `link_uid` → (`attr_out_tag`, `attr_in_tag`)

`attr_to_node`: `attr_tag` → `uid`

These three dicts are the 'model' in MVC terms.

The canvas and sidebar are the 'view'.

The callbacks are the 'controller'.

Understanding this separation is the foundation of your thesis architecture chapter.

4 Code Generation — From Canvas to Python File

4.1 Overview of the Pipeline

The `generate_code()` function in the prototype is a single-pass string builder. It walks the three data structures (`nodes`, `links`, `attr_to_node`) and emits a complete Python file in five stages.

```
Stage 1 : Collect import statements from each block's BlockDef
Stage 2 : Emit simulation parameters (T_SIM, DT)
Stage 3 : Emit block instantiation lines
           sin_src = SinusoidalGenerator('sin_src', amplitude=1, freq=50)
Stage 4 : Emit wiring lines
           sin_src >> gain
Stage 5 : Emit EmbedSim(), scope.add(), sim.run(), and plot calls
```

4.2 Stage 3 — Block Instantiation

For each `NodeInst` in `nodes.values()`, the generator:

1. **BlockDef** Looks up the
2. **nd.params** For each parameter, reads the current value from
3. **%g** Formats the value: floats use
4. **var = ClassName("var", param=value, ...)** Assembles the constructor call:

```
# Example: SinusoidalGenerator with amplitude=1, freq=50, phase=0
# BlockDef.params = [ParamDef('amplitude',1.0), ParamDef('freq',50.0), ...]
# NodeInst.params = {'amplitude': 1.0, 'freq': 50.0, 'phase': 0.0}

# Generated line:
sin_src = SinusoidalGenerator('sin_src', amplitude=1, freq=50, phase=0)
```

4.3 Stage 4 — Wiring

The wiring stage iterates `links.values()` and converts each (`attr_out_tag`, `attr_in_tag`) pair back to variable names via `attr_to_node` and `nodes`. An emitted set prevents duplicate lines when a block has multiple outputs going to the same target.

```
for attr_out, attr_in in links.values():
    src_uid = attr_to_node[attr_out] # which block owns this output pin
    dst_uid = attr_to_node[attr_in]  # which block owns this input pin
    src_name = nodes[src_uid].var_name
    dst_name = nodes[dst_uid].var_name
    emit: f'{src_name} >> {dst_name}'
```

4.4 What the Generated File Looks Like

The generated file shown in the uploaded example (`embedsim_generated.py`) demonstrates the format for a simple two-block diagram:

```
"""
Generated by EmbedSim GUI
Run: python <this_file>.py
"""
```



```
from embedsim.dynamic_blocks import VectorEnd
from embedsim.processing_blocks import VectorGain
from embedsim.simulation_engine import EmbedSim, ODESolver
from embedsim.source_blocks import SinusoidalGenerator

T_SIM = 3.0
DT     = 0.01

sin_src = SinusoidalGenerator("sin_src", amplitude=1, freq=50, phase=0)
gain     = VectorGain("gain", gain=1)
sink     = VectorEnd("sink")

sin_src >> gain
gain     >> sink

sim = EmbedSim(sinks=[sink], T=T_SIM, dt=DT, solver=ODESolver.HEUN)
sim.scope.add(sin_src, label="sin_src")
sim.scope.add(gain,    label="gain")
sim.run(verbose=False, progress_bar=True)

from embedsim.plot_helper import create_plotter
ph = create_plotter(sim)
ph.easypplot(signals=["sin_src[0]", "gain[0]"], title="EmbedSim GUI – Generated")
```

5 Running the Simulation

5.1 Current Workflow (Manual)

In the prototype, simulation is a two-step process: generate and save the .py file from the GUI, then run it in a terminal. The GUI does not execute any simulation code itself.

```
Step 1 – In the GUI: click 'Generate Code', then 'Save .py'
Step 2 – In a terminal:
$ python embedsim_generated.py
[Progress bar appears]
[Plot window opens after simulation completes]
```

This is functional but breaks the creative flow. Every time you want to see the effect of changing a gain value, you must save, switch to the terminal, run, wait, look at the plot, close it, switch back to the GUI, and change the value. For an iterative design process — which is what simulation is for — this friction is significant.

5.2 The Integrated Run Button — What Your Thesis Should Add

The most impactful single feature for user experience is an embedded "Run" button that executes the simulation inside the GUI and displays the results without leaving the application. This requires:

5. **generate_code()** Generating the code to a temporary file or string (already possible with
6. Executing it in a background thread (never in the GUI thread — that would freeze the interface).
7. **sim.scope** Capturing the
8. Rendering the scope data as plots embedded in the GUI.

```
import threading, io, contextlib

def _run_in_background(code_str: str, result_holder: dict):
    namespace = {}
    exec(compile(code_str, '<gui>', 'exec'), namespace)
    result_holder['sim'] = namespace.get('sim')

def cb_run(sender, app_data):
    code = generate_code(T, dt, solver)
    result = {}
    t = threading.Thread(target=_run_in_background,
                        args=(code, result), daemon=True)
    t.start()
    # poll t.is_alive() in a dpg.set_frame_callback loop
    # when done: render result['sim'].scope into a plot widget
```

5.3 Embedding Matplotlib Plots in Dear PyGui

Dear PyGui supports texture-based image rendering. A Matplotlib figure can be converted to a NumPy RGBA array and pushed to a Dear PyGui texture, making it appear as an inline image in the GUI window.

```
import matplotlib.pyplot as plt
import numpy as np

def mpl_to_dpg_texture(fig) -> tuple[int, int, list]:
```

```
fig.canvas.draw()
buf = fig.canvas.buffer_rgba()
arr = np.frombuffer(buf, dtype=np.uint8)
w, h = fig.canvas.get_width_height()
arr = arr.reshape(h, w, 4).astype(np.float32) / 255.0
return w, h, arr.flatten().tolist()

# Then in the GUI:
w, h, data = mpl_to_dpg_texture(fig)
with dpg.texture_registry():
    tex_id = dpg.add_dynamic_texture(w, h, data)
dpg.add_image(tex_id)
```

6 Architecture — Model · View · Controller

6.1 Why MVC Matters Here

The prototype works, but its architecture is implicit. The model (data), view (GUI), and controller (callbacks) are mixed in one 700-line file. This is acceptable for a prototype but becomes a maintenance problem as features are added.

Your thesis should articulate and implement a clean MVC separation. This is not just academic tidiness — it is the prerequisite for the web version (Section 8), where the Model and Controller must run identically whether the View is Dear PyGui or a browser.

6.2 The Model Layer

The model contains everything that describes a diagram, independently of any GUI framework. In the prototype this is scattered across module-level dicts. A proper model looks like:

```
# embedsim_gui/model.py
from dataclasses import dataclass, field
from typing import Dict, List, Tuple, Any

@dataclass
class ParamDef:
    name: str
    default: Any
    annotation: str = ''

@dataclass
class BlockDef:
    class_name: str
    module: str
    import_line: str
    color: Tuple[int,int,int,int]
    params: List[ParamDef]
    is_source: bool = False
    is_sink: bool = False
    is_loop_breaker: bool = False
    description: str = ''

@dataclass
class NodeInst:
    uid: int
    class_name: str
    var_name: str
    params: Dict[str, Any]
    pos: Tuple[int,int] = (200, 200)

@dataclass
class DiagramModel:
    nodes: Dict[int, NodeInst] = field(default_factory=dict)
    links: Dict[int, Tuple[int,int]] = field(default_factory=dict)
    attr_to_node: Dict[int,int] = field(default_factory=dict)
    block_registry: Dict[str, BlockDef] = field(default_factory=dict)

    def add_node(self, class_name, pos) -> NodeInst: ...
    def add_link(self, attr_out, attr_in) -> int: ...
    def remove_link(self, link_uid): ...
    def validate(self) -> List[str]: ... # loop checker
    def generate_code(self, T, dt, solver) -> str: ...
    def to_json(self) -> str: ... # diagram persistence
    def from_json(cls, json_str) -> 'DiagramModel': ...
```

6.3 The View Layer

The view is responsible only for displaying the model and forwarding user events to the controller. It must not modify the model directly. In a clean MVC design the view contains no business logic.

```
# embedsim_gui/view_dearpygui.py
class DearPyGuiView:
    def __init__(self, controller): ...
    def render_node(self, node_inst: NodeInst): ...
    def render_link(self, link_uid, attr_out, attr_in): ...
    def update_param_label(self, uid, pname, value): ...
    def show_sidebar(self, node_inst: NodeInst): ...
    def show_validation_result(self, errors: List[str]): ...
    def show_code(self, code_str: str): ...
    def run_event_loop(self): ...

# embedsim_gui/view_web.py ← your web extension
class WebSocketView:
    # same interface, different implementation
    def render_node(self, node_inst): ...
    # sends JSON over WebSocket to the browser
```

6.4 The Controller Layer

The controller receives user events from the view, updates the model, and instructs the view to refresh. Because it depends only on the Model and View interfaces, not their implementations, the same controller works for both Dear PyGui and the web view.

```
# embedsim_gui/controller.py
class DiagramController:
    def __init__(self, model: DiagramModel, view): ...

    def on_palette_drop(self, class_name, pos): ...
    # → model.add_node(class_name, pos)
    # → view.render_node(new_node)

    def on_link_created(self, attr_out, attr_in): ...
    # → model.add_link(attr_out, attr_in)
    # → view.render_link(...)

    def on_param_changed(self, uid, pname, value): ...
    # → model.nodes[uid].params[pname] = value
    # → view.update_param_label(uid, pname, value)

    def on_validate(self): ...
    # → errors = model.validate()
    # → view.show_validation_result(errors)

    def on_generate(self, T, dt, solver): ...
    def on_run(self, T, dt, solver): ...
    def on_save(self, path): ...
```

Layer	Responsibilities	What must NOT be there
Model	DiagramModel, NodeInst, BlockDef, DiagramModel.generate_code()	Any dpkg.* calls. Any HTML/JS. Any print() for UI purposes.

View	dpg.add_node(), dpg.add_text(), WebSocket.send(JSON)	Direct model mutation. Business logic. Loop detection.
Controller	Event handlers: on_palette_drop(), on_link_created() ...	dpg.* calls. HTML. Any rendering code.

7 The Block Registry — Adding New Blocks

7.1 The Current Manual Registry

In the prototype, every block that should appear in the palette must be manually added to `BLOCK_REGISTRY` — a Python list of `BlockDef` objects defined at the top of the file. If a new EmbedSim block is added to the framework (e.g. a Clarke transform block), nothing appears in the GUI until someone also edits `BLOCK_REGISTRY`.

This is the single most important maintenance burden in the prototype. For a framework with 30+ block types spread across several subpackages, keeping the registry in sync manually is error-prone.

7.2 Auto-Discovery via Python Inspection

Python's `inspect` module can walk an installed package and find all subclasses of `VectorBlock` automatically. This is the foundation of automatic block registration.

```
import inspect, importlib, pkgutil
import embedsim
from embedsim.core_blocks import VectorBlock

def discover_blocks() -> Dict[str, BlockDef]:
    registry = {}
    # Walk every module in the embedsim package
    for finder, mod_name, ispkg in pkgutil.walk_packages(
        path=embedsim.__path__, prefix='embedsim.'):
        try:
            module = importlib.import_module(mod_name)
        except ImportError:
            continue
        for name, cls in inspect.getmembers(module, inspect.isclass):
            if (issubclass(cls, VectorBlock) and
                cls is not VectorBlock and
                cls.__module__ == mod_name):
                registry[name] = _build_blockdef(name, cls, mod_name)
    return registry

def _build_blockdef(name, cls, module) -> BlockDef:
    sig = inspect.signature(cls.__init__)
    params = []
    for pname, p in sig.parameters.items():
        if pname in ('self', 'name'): continue
        default = p.default if p.default is not inspect.Parameter.empty else None
        params.append(ParamDef(pname, default))
    return BlockDef(class_name=name, module=module,
                    import_line=f'from {module} import {name}',
                    color=_color_for_module(module),
                    params=params)
```

7.3 Block Metadata via Class Attributes

For richer auto-registration (colour, category, description), block classes can carry metadata as class attributes. The auto-discovery function reads these if present, with sensible defaults if not.

```
class ClarkeTransformBlock(VectorBlock):
    TOPO_CATEGORY = 'transform' # → palette category
    TOPO_COLOR = (37, 99, 235, 255) # blue
    description = 'Clarke transform (abc → αβ)'
```

```
is_source    = False
is_sink      = False

# Auto-discovery reads these attributes if present:
color = getattr(cls, 'TOPO_COLOR', (100, 116, 139, 255))
desc  = getattr(cls, 'description', '')
cat   = getattr(cls, 'TOPO_CATEGORY', 'PROCESSING')
```

This is a one-time addition to each block class. After that, any new block added to EmbedSim appears automatically in the GUI palette the next time the GUI starts — no changes to the GUI code required.



For your thesis

Implement auto-discovery and demonstrate it by adding a new block to EmbedSim (e.g. a first-order low-pass filter block) and showing it appears in the GUI palette without any manual registry edit.

Measure: how many lines of GUI code change when a new block is added?

Before auto-discovery: O(10 lines) → After: O(0 lines).

This is the evidence for your 'extensibility' thesis claim.

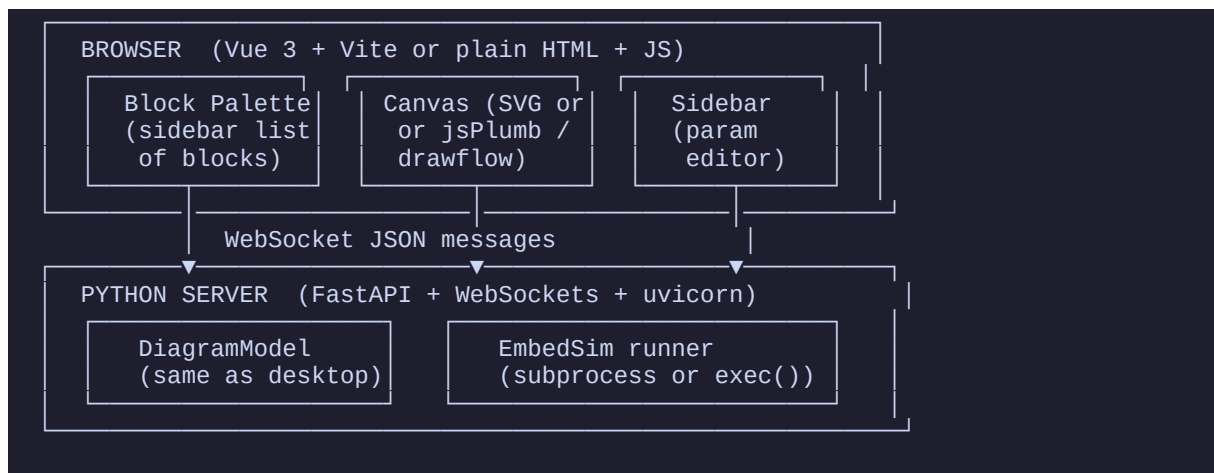
8 The Web Version — Design and Feasibility

8.1 Why a Web Version?

Dear PyGui requires a desktop Python installation — not available on mobile devices, Chromebooks, or locked-down corporate environments. A web version makes EmbedSim accessible to IoT engineering students on any device with a browser. This is a high-value thesis contribution for a project whose stated mission is "free access for engineers in India, Sri Lanka, and similar markets."

8.2 Architecture Choice: Python Backend + Browser Frontend

The cleanest approach runs the Python simulation code on a server (or locally via a minimal HTTP server), while the browser provides the visual interface. The two communicate over WebSockets.



8.3 Message Protocol

All communication uses JSON messages with a type field. The server sends the current diagram state on connection and broadcasts updates to all connected clients (for future multi-user collaboration).

```

# Client → Server
{ 'type': 'add_node',      'class_name': 'VectorGain', 'pos': [400, 200] }
{ 'type': 'add_link',     'attr_out': 1002, 'attr_in': 1005 }
{ 'type': 'update_param', 'uid': 3, 'pname': 'gain', 'value': 2.5 }
{ 'type': 'validate' }
{ 'type': 'run',          'T': 3.0, 'dt': 0.01, 'solver': 'HEUN' }

# Server → Client
{ 'type': 'state',        'nodes': [...], 'links': [...] }
{ 'type': 'node_added',   'node': {...} }
{ 'type': 'validate_result', 'errors': [] }
{ 'type': 'run_progress', 'percent': 42 }
{ 'type': 'run_complete', 'plot_data': { 't': [...], 'signals': {...} } }
  
```

8.4 The Canvas Library Choice

The hardest part of the web frontend is the interactive node-editor canvas. Several JavaScript libraries provide this:

Library	Licence / Cost	Node-editor support	Recommendation
Drawflow	MIT — free	Built-in, lightweight	Best for a thesis prototype — simple, well-documented
React Flow	MIT — free	Excellent, React-based	Best if you use React; more complex setup
jsPlumb	Community: MIT	Good, but older API	Acceptable alternative
Custom SVG/Canvas	N/A	Full control, more work	Only if library constraints are unacceptable

8.5 Minimal Web Server (FastAPI)

```
# embedsim_web_server.py
from fastapi import FastAPI, WebSocket
from fastapi.staticfiles import StaticFiles
import json, asyncio

app = FastAPI()
app.mount('/static', StaticFiles(directory='static'), name='static')

model = DiagramModel() # same model as desktop
controller = DiagramController(model, WebSocketView())

@app.websocket('/ws')
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()
    await ws.send_text(json.dumps({'type': 'state', **model.to_dict()}))
    async for msg in ws.iter_text():
        data = json.loads(msg)
        await controller.handle_message(data, ws)

# Run: uvicorn embedsim_web_server:app --reload
# Open: http://localhost:8000/static/index.html
```

9 Thesis Improvements — What to Build and Why

9.1 Improvement A — MVC Refactor (Foundational, Must Do)

Without the MVC refactor, every subsequent improvement is building on sand. The MVC refactor is not just organisational tidiness — it is the prerequisite that makes the web version possible without rewriting the business logic.

Your thesis should document the refactor with a before/after comparison: lines of code in the monolithic prototype vs the separated layers, and a test showing that the same DiagramModel generates identical code regardless of whether the view is Dear PyGui or a mock.

9.2 Improvement B — Diagram Persistence (Save/Load JSON)

Currently the only way to "save" your work is to generate the .py file. If you want to come back and edit the diagram, you have to rebuild it from scratch. A JSON serialisation of the DiagramModel allows save/load of the canvas itself.

```
# DiagramModel.to_dict() / from_dict() (schematic JSON)
{
  "nodes": [{ "uid":1, "class":"SinusoidalGenerator", "var":"sin_src",
    "params":{"amplitude":1, "freq":50}, "pos":[80,200]}, ...],
  "links": [{ "uid":5001, "attr_out":1002, "attr_in":1005}, ...]
}

# This JSON is version-controlled alongside the simulation .py file.
# A team can share diagrams via git, not just generated code.
```

9.3 Improvement C — Auto-Discovery (Section 7, High Value)

Implement the `discover_blocks()` function from Section 7. Measure and report: how many GUI code changes are required when a new EmbedSim block is added? Before: 10+ lines. After: 0. This is a concrete, measurable improvement with a clear before/after story.

9.4 Improvement D — Integrated Run + Inline Plots

Implement the background simulation runner from Section 5.2. The user experience improvement is dramatic: the edit-run-observe cycle drops from 6 manual steps to 1 button click. Measure the time-to-insight for a representative task ("change the gain and observe the effect on the output") with and without the integrated runner.

9.5 Improvement E — Web Version (Section 8)

The web version is the most ambitious but also the highest-impact thesis contribution. A working prototype needs only three components: the FastAPI WebSocket server, the DiagramModel (reused from the desktop version), and a minimal Drawflow-based HTML frontend.

A minimum viable web demo can be built in approximately 200 lines of Python (server + controller) and 150 lines of HTML/JavaScript (frontend). This is achievable in one week once the MVC refactor is done.

#	Improvement	Prerequisite	Effort	Thesis impact
A	MVC refactor	None	3–5 days	Foundational — enables B–E
B	Save/load diagram JSON	A	1–2 days	High usability gain
C	Auto-discovery of blocks	A	2–3 days	Eliminates manual registry
D	Integrated run + inline plots	A	3–5 days	Largest UX improvement
E	Web version (FastAPI + Drawflow)	A, B	1–2 weeks	Highest reach — any device

10 Deliverables and Thesis Claims

Deliverable	Contents	Evidence for which claim
<code>embedsim_gui/model.py</code>	DiagramModel, NodeInst, BlockDef — no GUI imports	MVC separation, testability
<code>embedsim_gui/view_dpg.py</code>	DearPyGuiView — all dpg.* calls isolated here	MVC separation
<code>embedsim_gui/controller.py</code>	DiagramController — handles all user events	MVC separation
<code>embedsim_gui/discovery.py</code>	discover_blocks() — auto-registration via inspect	Extensibility claim
<code>embedsim_web_server.py</code>	FastAPI + WebSocket server, reuses DiagramModel	Web accessibility claim
<code>static/index.html + app.js</code>	Browser frontend: Drawflow canvas + JSON protocol	Web accessibility claim
<code>tests/test_model.py</code>	Unit tests: add_node, add_link, validate, generate_code	Code quality, reproducibility
<code>user_study/results.csv</code>	Task-completion times: prototype vs improved GUI	Usability claim

CLAIM 1 — MVC Separation

"The refactored GUI achieves strict MVC separation: DiagramModel has zero imports of Dear PyGui or any web framework. The same model generates identical Python code whether the view is the desktop or web frontend."

Evidence: import analysis of model.py; unit test running generate_code() without any GUI.

CLAIM 2 — Extensibility

"Adding a new EmbedSim block requires zero changes to the GUI codebase. The block appears automatically in the palette and code generator on the next application start."

Evidence: add a new block class to EmbedSim; restart the GUI; show the block in the palette.

CLAIM 3 — Web Accessibility

"The web version runs the full block-diagram workflow — palette, wiring, validate, generate, run — in a browser with no local Python installation required,

on a server instance accessible via URL."

Evidence: screenshot or video of the web UI; URL of the running demo.

11 Your First Week — Day by Day

Monday — Read and Reproduce

9. **pip install dearpygui** Install Dear PyGui:
10. **python embedsim_gui.py** Run the prototype:
11. **embedsim_generated.py** Read the generated
12. **nodes** Find the three data structures (

Tuesday — The >> Operator

13. Open a Python REPL. Import EmbedSim. Write a simple three-block simulation by hand. Run it. Understand what each >> call does.
14. Add a VectorSum block. Wire two sources to it using >>. What happens if you wire in the wrong order?
15. Introduce an algebraic loop deliberately. Observe the error. Fix it with VectorDelay. Run the GUI validator on the same diagram and confirm it catches the loop.

Wednesday — Start the MVC Refactor

16. **embedsim_gui**/Create
17. **BlockDef**Move
18. Write one unit test: create a DiagramModel with two nodes and a link; call generate_code(); assert the output contains the >> wiring line.

Thursday — Auto-Discovery

19. **discover_blocks()** Implement
20. Print the discovered block list. Compare it with the manual BLOCK_REGISTRY. Are they the same?
21. Replace BLOCK_REGISTRY with the auto-discovered dict in the GUI. Verify the palette still shows all blocks.

Friday — Diagram Persistence

22. **DiagramModel.to_dict()** Implement
23. Add Save Diagram and Load Diagram buttons to the toolbar.
24. **my_diagram.json** Test: build a diagram, save it as

You are not building a drawing tool.

You are building the interface between a human idea and a running simulation.

Every friction point you remove is one more engineer who can use EmbedSim.

R Reference: Key Classes, Methods and Libraries

R.1 Key Classes in the Prototype

Class / Function	Location	Purpose
BlockDef	embedsim_gui.py (top)	Descriptor for one EmbedSim block type
NodeInst	embedsim_gui.py	One placed block on the canvas — uid, class, params, pin tags
generate_code()	embedsim_gui.py	Walks nodes+links dicts → emits complete .py file
validate_loops()	embedsim_gui.py	DFS cycle detection, reports algebraic loops
add_node()	embedsim_gui.py	Creates NodeInst + dpg node widget from a class name
cb_link_created()	embedsim_gui.py	Callback when user draws a wire: stores in links dict
show_sidebar()	embedsim_gui.py	Builds parameter editor widgets for selected node

R.2 Libraries and Frameworks

Library	Version	Role
dearpygui	1.11+ (Python 3.12)	Desktop GUI: node editor, widgets, themes. Install: pip install dearpygui
FastAPI	0.111+	Web backend with WebSocket support. Install: pip install fastapi uvicorn
Drawflow	0.0.59 (JS)	Browser node-editor canvas. CDN: cdn.jsdelivr.net/npm/drawflow
matplotlib	3.8+	Plot generation — convert to NumPy array for DPG texture embedding
pkgutil / inspect	stdlib	Block auto-discovery — walk package, find VectorBlock subclasses
pytest	7+	Unit tests for model layer. No GUI required to run tests.

R.3 Key Papers and Standards

Reference	Why it matters
Nielsen, J. (1994) — Usability Engineering, Morgan Kaufmann	The 10 usability heuristics — framework for your user study
Fowler, M. (2002) — Patterns of Enterprise Application Architecture	MVC pattern canonical reference
ISO 9241-210:2019 — Human-Centred Design for	Standard for user-centred design process —

Interactive Systems	cite in your methodology
Dear PyGui documentation — dearpygui.readthedocs.io	Node editor API reference — <code>mvNode_Attr_Input</code> , <code>mvNodeMiniMap_*</code> , etc.
FastAPI documentation — fastapi.tiangolo.com	WebSocket endpoint design, async patterns
EmbedSim — github.com/vectorsim/embed_sim_project	The framework your GUI drives — all <code>VectorBlock</code> subclasses documented here

EmbedSim · Embedded Control · Open Source · github.com/vectorsim/embed_sim_project